

ECR (electronic cash registrar) протокол ПриватБанк (JSON based)

Содержание:

1. Описание физического уровня протокола.
2. Описание канального протокола передачи данных.
3. Архитектура работы. Разработка клиентского приложения.
4. Прикладной уровень.
5. Методы
 - 5.1 Метод «Оплата»
 - 5.2 Операция «Возврат»
 - 5.3 Операция «Отмена»
 - 5.4 Операция «Частичная отмена»
 - 5.5 Операция «Проверка связи»
 - 5.6 Операция «Чтение банковской карты»
 - 5.7 Операция «Чтение дисконтной карты»
 - 5.8 Операция «Версия ПО»
 - 5.9 Операция «Ping»
 - 5.10 Операция «Баланс»
 - 5.11 Операция «Сервис возврат»
 - 5.12 Операция «Сервис оплата частями» (Pay by parts)
 - 5.13 Операция «Сервис возврат оплаты частями»
 - 5.14 Прочие сервисы
 - 5.15 Универсальный сервис
 - 5.16 Cashback
 - 5.17 Audit (X – balance)
 - 5.18 Verify (Загальний звіт)
 - 5.19 Копия чека «Сверка»

5.20 Получение чека

5.21 Печать информации о всех транзакциях в пакете

6. Служебные сообщения

6.1 Сообщение deviceBusy

6.2 Сообщение interrupt

6.2.1 Ответ на сообщение interrupt – подтверждение interruptTransmitted.

6.3 Сообщение methodNotImplemented

6.4 Сообщение getMerchantList

6.4 Сообщение getMaskList

6.5 Сервисное сообщение debug

6.6 Service message getLastResult

6.7 Service message getLastStatMsgCode

6.8 Service message getLastStatMsgDescription

6.9 Service message Identify

6.10 Service message getDiscountName

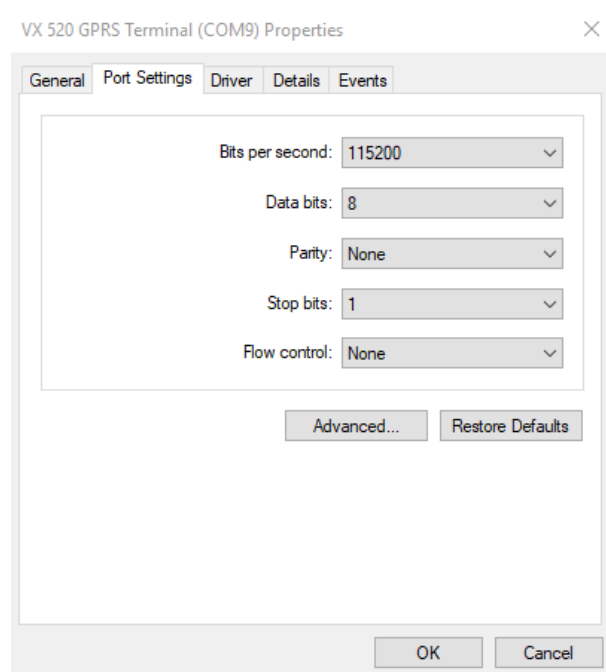
6.11 Service message correctTransaction

6.11.1 Ответ на сообщение correctTransaction - подтверждение correctionTransmitted

1. Описание физического уровня протокола

Терминал, в зависимости от аппаратных возможностей конкретной модели, подключается к кассовому аппарату (либо эмулирующему программному обеспечению) через USB порт, аппаратный COM порт либо посредством Ethernet/WIFI (IEEE 802.x).

При наличии различных интерфейсов одновременно, на стороне терминала предусмотрена возможность переключения между ними путем настройки совмещения на стороне банка. Для OS семейства Windows USB-подключение подразумевает установку стандартного драйвера вендора. При этом в системе создаётся виртуальный COM порт (usbserial bridge). Формула настроек COM порта следующая: **115200, 8N1**.



Для Linux используется индивидуальная настройка COM/USB. В качестве общих рекомендаций, при подключении по USB необходимо использовать стандартный usbserial драйвер. В случае не определения, можно указать id устройства вручную:

```
modprobe usbserial vendor=0x11ca product=0x0222 (для Verifone vx520)
```

Также полезно отключить/удалить modemmanager, что существенно ускорит определение/переопределение устройства UDEV. При желании, можно прописать и собственные правила UDEV.

Не исключается работа клиентского приложения с USB напрямую, по выбору интегратора. Для этого рекомендуется использование библиотеки **libusb**. Данная библиотека - кроссплатформенная.

Работа через сетевые интерфейсы подразумевает использование протокола TCP. Порт по умолчанию - 2000. В настройках терминала имеется возможность изменить номер порта для кассового совмещения, а также включить/отключить DHCP.

Описывать собственно физику работы последовательных портов в данном документе не видится целесообразным, поскольку низкоуровневое программирование устройств с использованием сигналов DTE/DCE, прямого программирования портов, прерываний и т. п. давно уступило место программированию с использованием высокоуровневых языков, оперирующих абстракциями типа Open, Close, Read, Write. Конкретная реализация будет зависеть от выбранного языка программирования и операционной системы. Специальных команд прикладного уровня для управления открытием и закрытием соединения не требуется в том смысле, что программист самостоятельно управляет открытием и закрытием порта и отправка JSON сообщения для этого не требуется.

2. Описание канального протокола передачи данных

Основа данного протокола (далее - Протокол) - формат обмена данными JSON. Для упрощения обмена данными, байтовые последовательности JSON завершаются символами-терминаторами (делимитерами) 0x00 (как строки в языке C). Применение делимитера призвано служить заменой управляющим ASCII кодам, применяемым в подобных протоколах. Протокол един для всех типов подключения. Вследствие данной установки, в случае работы с последовательным портом, он выполняет роль протокола канального уровня. В случае IEEE 802.x, поскольку дейтаграммы упаковываются в TCP/IP, Протокол становится прикладным, не меняясь по содержанию.

Важно!

При использовании сетевого типа подключения, необходимо учитывать следующее:

- Терминал не является полноценным сетевым сервером. В Протоколе отсутствует управление доступом и предотвращение коллизий.
- Для реализации подобного функционала необходимо разрабатывать собственное ПО - транслятор в websocket, HTTP(S), другой выбранный протокол/транспорт.
- Существует эталонная схема подключения клиента к терминалу, которая используется в эмуляторе кассы ПриватБанк:
 1. Коннект на IP, порт
 2. Отправляем хэндшейк (см. далее), проверяем ответ
 3. Дисконнект
 4. Отправляем Identify(см. далее), проверяем ответ (возможно ветвление кода, согласно полученным в ответе наименованиям вендора и модели терминала)
 5. Дисконнект
 6. Далее - основной режим работы: открываем соединение, держим его открытым (keepalive) и слушаем входящие данные ПОСТОЯННО, разделяя дейтаграммы по символу 0x00
 7. Полученный на этапе 6 дескриптор соединения, храним, как глобальный, и используем его для чтения, по мере надобности. НЕ ОТКЛЮЧАЕМСЯ
 8. При желании, производится проверка соединения и переподключение при обнаружении обрыва (см. пункт 1)
- Аналогичная по структуре схема применяется и при работе через USB/COM
- Данная схема проходит тестирование и гарантирует работу с терминалами любых вендоров, поддерживающих Протокол. В основе схемы - постоянный монопольный доступ с поддержкой соединения и переподключением при обрыве.
- Другие способы работы возможны, но не гарантируются.

Важно!

В качестве значений того или иного поля JSON все не-ASCII символы в т. ч. кириллица, передаются в кодировке utf-8

Например:

Исходный JSON string:

```
{"method":"GetTerminalInfo","step":0}
```

Исходный JSON [] byte:

0x7b, 0x22, 0x6d, 0x65, 0x74, 0x68, 0x6f, 0x64, 0x22, 0x3a, 0x22, 0x47, 0x65, 0x74, 0x54, 0x65, 0x72, 0x6d, 0x69, 0x6e, 0x61, 0x6c, 0x49, 0x6e, 0x66, 0x6f, 0x22, 0x2c, 0x22, 0x73, 0x74, 0x65, 0x70, 0x22, 0x3a, 0x30, 0x7d

NULL-terminated JSON:

0x7b, 0x22, 0x6d, 0x65, 0x74, 0x68, 0x6f, 0x64, 0x22, 0x3a, 0x22, 0x47, 0x65, 0x74, 0x54, 0x65, 0x72, 0x6d, 0x69, 0x6e, 0x61, 0x6c, 0x49, 0x6e, 0x66, 0x6f, 0x22, 0x2c, 0x22, 0x73, 0x74, 0x65, 0x70, 0x22, 0x3a, 0x30, 0x7d, 0x00

Процедура хэндшейка

Процедура хэндшейка осуществляется посредством передачи сообщения **Ping**:

{"method":"PingDevice","step":0}

Однако данное сообщение отличается дополнительным терминатором 0x00 в начале передаваемой последовательности:

Written data (port):

00 7b 22 6d 65 74 68 6f 64 22 3a 22 50 69 6e 67
44 65 76 69 63 65 22 2c 22 73 74 65 70 22 3a 30
7d 00

Ответ на данное сообщение:

{"method":"PingDevice","step":0,"params":{"code":"00","responseCode":"0000"},"error":false,"errorDescription":""}.

Read data (port):

7b 22 6d 65 74 68 6f 64 22 3a 22 50 69 6e 67 44
65 76 69 63 65 22 2c 22 73 74 65 70 22 3a 30 2c
22 70 61 72 61 6d 73 22 3a 7b 22 63 6f 64 65 22
3a 22 30 30 22 2c 22 72 65 73 70 6f 6e 73 65 43
6f 64 65 22 3a 22 30 30 30 30 22 7d 2c 22 65 72
72 6f 72 22 3a 66 61 6c 73 65 2c 22 65 72 72 6f
72 44 65 73 63 72 69 74 70 74 69 6f 6e 22 3a 22
22 7d 00

В результате такого обмена устройство считается обнаруженным, процедура хэндшейка - пройденной.

Итого запрос-ответ:

```
1 Written data (port):
2
3 00 7b 22 6d 65 74 68 6f 64 22 3a 22 50 69 6e 67
4 44 65 76 69 63 65 22 2c 22 73 74 65 70 22 3a 30
5 7d 00
6
7 {"method":"PingDevice","step":0}.
8
9
10 Read data (port):
11
12 7b 22 6d 65 74 68 6f 64 22 3a 22 50 69 6e 67 44
13 65 76 69 63 65 22 2c 22 73 74 65 70 22 3a 30 2c
14 22 70 61 72 61 6d 73 22 3a 7b 22 63 6f 64 65 22
15 3a 22 30 30 22 2c 22 72 65 73 70 6f 6e 73 65 43
16 6f 64 65 22 3a 22 30 30 30 30 22 7d 2c 22 65 72
17 72 6f 72 22 3a 66 61 6c 73 65 2c 22 65 72 72 6f
18 72 44 65 73 63 72 69 74 70 74 69 6f 6e 22 3a 22
19 22 7d 00
20
21 {"method":"PingDevice","step":0,"params":{"code":"00","responseCode":"0000"},"error":false,"errorDescription":""}.
22
23 Close port.
```

Все прочие дейтаграммы передаются с NULL-терминатором только в конце JSON сообщения. Сами JSON сообщения передаются в виде бинарного протокола как последовательности байтов.

Identify (рекомендовано использовать после хэндшейка)

Пример обмена данными:

{"method":"ServiceMessage","step":0,"params":{"msgType":"identify"}}

Data to send: =====

7b 22 6d 65 74 68 6f 64 22 3a 22 53 65 72 76 69 63 65 4d 65 73 73 61 67 65 22 2c 22 73 74
65 70 22 3a 30 2c 22 70 61 72 61 6d 73 22 3a 7b 22 6d 73 67 54 79 70 65 22 3a 22 69 64 65
6e 74 69 66 79 22 7d 7d 00

=====

**{"method": "ServiceMessage", "step": 0, "params": {"msgType": "identify", "result":
"true", "vendor": "PAX", "model": "s800"}, "error": false, "errorDescription": ""}**

RCV: =====

7b 22 6d 65 74 68 6f 64 22 3a 20 22 53 65 72 76 69 63 65 4d 65 73 73 61 67 65 22 2c 20 22
73 74 65 70 22 3a 20 30 2c 20 22 70 61 72 61 6d 73 22 3a 20 7b 22 6d 73 67 54 79 70 65 22
3a 20 22 69 64 65 6e 74 69 66 79 22 2c 20 22 72 65 73 75 6c 74 22 3a 20 22 74 72 75 65 22
2c 20 22 76 65 6e 64 6f 72 22 3a 20 22 50 41 58 22 2c 20 22 6d 6f 64 65 6c 22 3a 20 22 73
38 30 30 22 7d 2c 20 22 65 72 72 6f 72 22 3a 20 66 61 6c 73 65 2c 20 22 65 72 72 6f 72 44
65 73 63 72 69 70 74 69 6f 6e 22 3a 20 22 22 7d 00

=====

3. Архитектура работы. Разработка клиентского приложения.

Разработка клиентского приложения необходима для реализации подключения кассы непосредственно к терминалу с использованием Протокола, а также, возможно, для имплементации дополнительного функционала, к примеру, протокола WebSocket, HTTP(S) и т. п.

Сервер может состоять из пишущей и слушающей части, работающих в дуплексе.

Слушающая часть (Listener) поддерживает **ПОСТОЯННОЕ** подключение к терминалу и формирует буфер данных по достижению NULL – терминатора (делимитера). При потере соединения - производится попытка его восстановления.

Пример (на языке Go):

```
//Listener carries out a permanent TCP connection to get raw data from
terminal (upon receiving).
//Removes delimiter (0x00). Sends data to channel (BuffHandler() func is the
receiver).
//Carries out disconnections
//Must be used as a goroutine
func Listener() {

    var err error

    Gconn, err = net.Dial("tcp", GconnOption)
    if err != nil {
        log.Fatalf("net.Dial: %v", err)
    }

    log.Println("Listening Network...")

    answ := make([]byte, 1024)

    buf := bytes.Buffer{}

    for {
        i := 0
        i, err = Gconn.Read(answ)
        if err != nil && err != io.EOF {
            log.Printf("conn.Read: %v", err)
            Gconn.Close()
            go reconnect() // reconnect
            break
        }
        if i != 0 {
            answ = answ[:i]
            buf.Write(answ)
            if buf.Bytes()[len(buf.Bytes())-1] == 0x00 { //delimiter
                trace(buf.Bytes()) //if debug, write trace
                a := buf.Bytes()[:len(buf.Bytes())-1] //remove delimiter
                buffer <- a
                buf.Reset()
            }
        }
    }
}
```

Обработчик (с передачей данных в websocket)

```
//BuffHandler gets data via channel from the Listener func (upon receiving).
//Transmits terminal answers to websocket.
//Must be used as goroutine.
func BuffHandler() {
    for {
        buf := <-buffer
        conn := *wsconn
        err := conn.WriteMessage(websocket.TextMessage, buf)
        if err != nil {
            log.Println("Error in BuffHandler: ", err)
        }
    }
}
```

Writer

```
//prepare method ping
if getJSON.Method == "PingDevice" && getJSON.Step == 0 && !busy {

    go func() {

        var p jsonEntityNoParams
        p.Method = "PingDevice"
        p.Step = 0

        data, err := json.Marshal(p)
        if err != nil {
            log.Println("Error marshaling JSON: ", err)
            return
        }
        data = append(data, 0x00)// add terminator

        err = writeEth(data)
        if err != nil {
            log.Printf("conn.Write: %v", err)
        }
    }()
}

//writer
func writeEth(buffer []byte, timeout ...int) error {

    var n int
    var err error

    var t Stopwatch
    t.Start() // start timer
    defer t.Stop() // sure to stop timer

    //set timeout
    var tout int
    if len(timeout) == 0 {
        tout = 30
    } else {
        tout = timeout[0]
    }

    if connIsClosed() {
        if err = Gconn.Close(); err == nil {
```

```

        err = errors.New("disconnection")
    }
    return err
}

for {
    if err = Gconn.SetWriteDeadline(time.Now().Add(5 * time.Second)); err !=
nil {
        continue
    }
    n, err = Gconn.Write(buffer)
    if err != nil {
        log.Println("Error in WriteEth(): ", err)
        sleep(1)
    } else {
        break
    }
    if t.GetElapsed() >= time.Duration(tout)*time.Second {
        err = errors.New("connection timeout")
        break
    }
}

if err != nil {
    log.Printf("TCP write: %v", err)
    err = errors.New("write error")
}

fmt.Println("Wrote", n, "bytes.")

trace(buffer) // trace with delimiter NB!

return err
}

```

Методы категории **ServiceMessage** являются асинхронными и могут быть выполнены в любой момент времени, даже в момент выполнения другого метода. Прочие методы перехватывают процесс и до своего завершения возвращают **ServiceMessage Busy**. Таким образом, при реализации интегратором работы с терминалом, необходимо понимать, что устройство должно работать с 3 логическими потоками данных

1. Текущая операция
2. Обмен Service Message
3. В ответ на конкурирующий запрос, не относящийся к категории сервисных, ответ: ServiceMessage Busy.

Сказанное выше не означает, что к терминалу можно обращаться с разных касс параллельно. Описанные 3 логических потока существуют в рамках 1 коннекта. Для осуществления запроса от второго, третьего... N клиента нужно полностью освободить терминал и проводить каждый раз процедуру хэндшейка.

Данные особенности должны всегда учитываться как при обмене данными между кассой и терминалом в монопольном режиме, так и при разработке middleware-средств для организации множественного доступа к терминалу.

Полный перечень ServiceMessage:

deviceBusy

interrupt

interruptTransmitted (существует только как ответ терминала)

methodNotImplemented (существует только как ответ терминала)

getMerchantList

getMaskList

debug (реализуется на стороне клиента!)

getLastResult

getLastStatMsgCode

getLastStatMsgDescription

identify

4. Прикладной уровень

Взаимодействие посредством JSON

Не смотря на наличие более производительных и современных альтернатив, к примеру таких, как Protocol Buffers, предложенный Google, с точки зрения распространенности, совместимости и удобства имплементации, формат JSON является на данный момент фактическим стандартом формата сериализации (передачи) структурированных данных.

В общем виде команды для терминала выглядят следующим образом:

```
{
  "method": "DoSomething",
  "step": 0,
  "params": {
    "par1": "xxx",
    "parN": "yyy"
  }
}
```

Ответ на этот запрос:

```
{
  "method": "DoSomething",
  "step": 0,
  "params": {
    "answer": "OperationResults"
  },
  "error": true,
  "errorDescription": "TIMEOUT"
}
```

Данный JSON приведен к единому типу и легко десериализуется:

C#:

=====

```
public class Params
{
    public string answer { get; set; }
}
```

```
public class RootObject
{
    public string method { get; set; }
    public int step { get; set; }
    public Params @params { get; set; }
    public bool error { get; set; }
    public string errorDescription { get; set; }
}
```

=====

GO:

=====

```
type jsonEntity struct{
    Method string `json:"method"` // tag descriptors
    Step int `json:"step"`
    Params map[string]string `json:"params"`
    Error bool `json:"error"`
    ErrorDescription string `json:"errorDescription"`
}
```

=====

Эмулятор кассового аппарата, пример

:

Данные

OPEN

RESPONSE: {"method":"PingDevice","step":0,"params":{"code":"00"},"error":false}

Устройство подключено

RESPONSE: {"method":"GetTerminalInfo","step":0,"params":{"code":"00","version":"T5PRVa_03K TKASPRBK80321-922-618"},"error":false}

Firmware: T5PRVa_03K TKASPRBK80321-922-618

method — название проводимой операции

step — зарезервировано для многопроходных операций (в однопроходных операциях всегда равен «0»)

params — массив дополнительных параметров для терминала

error — присутствие ошибки (true/false)

errorDecription — описание ошибки

Важно!

В данной реализации API ошибки всегда описываются двумя полями:

- поле **error** имеет тип **bool** и, соответственно, может принимать значения **true** и **false**.
- поле **errorDescription** - текстовое, и содержит описание ошибки и/или её код.

При прямом подключении кассы к терминалу, то есть без использования ПО для совмещения, таймауты, обрывы и прочие ошибки связи перехватываются на соответствующем уровне порта или сетевой модели, и обрабатываются интегратором на программном уровне.

Важно!

При **responseCode** $\geq$ 1000, а именно:

1000 – General error (should be used in exceptional case)
1001 – Transaction canceled by user
1002 – EMV Decline
1003 – Transaction log is full. Need close batch
1004 – No connection with host
1005 – No paper in printer
1006 – Error Crypto keys
1007 – Card reader is not connected
1008 – Transaction is already complete

Ответ приходит в сокращенном виде (без чека и прочих атрибутов транзакции)

```
{  
  "method": "Purchase",  
  "step": 0,  
  "params": {  
    "responseCode": "1001"  
  },  
  "error": true,  
  "errorDescription": "Transaction canceled by user"  
}
```

При некоторых **responseCode**, к примеру, 05, 96 и т. п. на кассу передаётся чек, но отсутствует ряд параметров, таких как **RRN**, **ApprovalCode** и т. д. В таких случаях ответ терминала и чек должны соответствовать таковым в ВПОС.

При **responseCode** отличных от 0000 терминал передаёт **"error":true**

Purchase, код 5, пример (Verifone):

```
{
  "method": "Purchase",
  "step": 0,
  "params": {
    "receipt": "ПРИВАТБАНК\nDNIPRO\nSALECPL\nMUDROGO 5
12222\n€\€\____\Ж\Ж\Ж\Ж\Ж3333\n\nВ І Д Х И Л Е Н
О\n\nАВТОРИЗАЦІЮ ВІДХИЛЕНО!\nЗВЕРНІТЬСЯ В БАНК-ЕМІТЕНТ
КАРТКИ\nКОД 05\n\n\n\n\nMASTER \n*****9732\nMasterCardAID:
A0000000041010\nТЕРМІНАЛ # S11103W202 ЖОВ 2019 16:38:11\nКОД
АВТОРИЗ.: DECLINEЧЕК N: 051886\n\n(c)SSI 2018 PRIVATBANK
v.T5PRVa_.03K\nSmartPos_EMV 01.001",
    "responseCode": "0005"
  },
  "error": true,
  "errorDescription": "ВІДХИЛЕНО! ЗВЕРНІТЬСЯ В БАНК-ЕМІТЕНТ КАРТКИ"
}
```

Важным отличием данного протокола является отсутствие подтверждения и выдачи чека на кассу как отдельных сущностей. В ситуации, где подразумевается confirm - используется автоподтверждение. Все операции, подразумевающие возврат чека на кассу, используют формат запрос-ответ, минуя отдельное требование выдачи чека.

5. Методы

5.1 Метод «Оплата»

Это основная операция терминала, предназначена для оплаты товара.

5.1.1 Запрос на оплату:

```
{  
  "method": "Purchase",  
  "step": 0,  
  "params": {  
    "amount": "0.60",  
    "discount": "",  
    "merchantId": "0",  
    "facepay": "false", // true  
    "subMerchant": "i12345678910"  
  }  
}
```

amount - сумма проводимой транзакции,

discount - дополнительная сумма (скидка)

merchantId - индекс мерчанта, который используется для транзакции

facepay - параметр запускающий с кассы оплату через FacePay24 (для Android терминалов с поддержкой данного сервиса)

Важно! В поле Amount допускается разделение суммы (грн/коп) только через знаки "." или ",",

ПО терминала при получении от кассы в поле amount любых символов отличных от "." или ",", должно прерывать продажу и возвращать на кассу ответ:

"responseCode": "1000"

"errorDescription": "Операція неможлива"

Сумма предоставляемая в формате вещественного числа приводится к строке, количество разрядов в дробной части - 2

0 -> "0.00"

5.1.2 Ответ на запрос

Терминал обработает транзакцию как обычную карточную операцию и передаст кассе код ответа, полученный от хоста. После связи с хостом терминал генерирует сообщение, приведённое ниже. Если касса печатает чеки, терминал пришлёт ей все необходимые данные, в противном случае будут отосланы только поле «**responseCode**», остальные же будут пустыми. Включение печати чеков кассой или терминалом настраивается в профиле терминала.

```
{
  "method": "Purchase",
  "step": 0,
  "params": {
    "amount": "0.60",
    "approvalCode": "999999",
    "captureReference": "",
    "cardExpiryDate": "2020",
    "cardHolderName": "INSTANT/ISSUE",
    "date": "02.10.2019",
    "discount": "0.00",
    "hstFld63Sf89": "",
    "invoiceNumber": "999999",
    "issuerName": ""VISA ПРИВАТ",
    "merchant": "TSTTTTTT",
    "pan": "4731XXXXXXXXX9838",
    "posConditionCode": "00",
    "posEntryMode": "022",
    "processingCode": "000000",
    "receipt": "text-of-receipt",
    "responseCode": "0000",
    "rrn": "99999999999999",
    "rrnExt": "88888888888888",
    "terminalId": "TSTSALE2",
    "time": "09:11:07",
    "track1": "",
    "signVerif": "0",
    "txnType": "1",
    "trnStatus": "1",
    "adv": "ПриватБанк.",
    "adv2p": "Беремо і робимо!",
```

```

    "bankAcquirer": "ПриватБанк",
    "paymentSystem": "VISA",
    "subMerchant": "i12345678910"
  },
  "error": false,
  "errorDescription": ""
}

```

responseCode — код ответа терминала
approvalCode — код авторизации
amount — сумма проводимой транзакции,
discount — дополнительная сумма (скидка),
pan — номер карты с учётом контрольной цифры
date — дата транзакции
time — время транзакции
cardHolderName — имя картодержателя
invoiceNumber — номер чека подтверждённой транзакции
issuerName — имя эмитента, по которому производилась операция
merchant — номер мерчанта, по которому производилась операция
posEntryMode — метод ввода карты
processingCode — код обработки
captureReference — 17 поле в сообщении сервера
cardExpiryDate — срок годности карты
hstFld63Sf89 — данные с хоста (63 поле, 89 подполе)
posConditionCode — код состояния терминала
receipt — чек
rrn — уникальный номер транзакции
rrnExt — идентификатор транзакции в банке эмитенте карты (приходит если карта не ПриватБанка)
terminalId — идентификатор терминала на хосте
track1 — первый трек карты
signVerif — проверка подписи (CVM)
txnType — тип текущей транзакции (*значения согласно принятым в BPOS)
trnStatus — статус транзакции (**значения согласно принятым в BPOS)
adv — первая хостовая реклама
adv2p — вторая хостовая реклама
bankAcquirer — наименование банка-эквайера
paymentSystem — наименование МПС передающееся с хоста
subMerchant — Субмерчант

*- *get_TxnType(BYTE* pVal), property TxnType*
[OUT] pVal – gets transaction type of current transaction. Intended for usage for GetTxnDataByInv and GetTxnDataByOrder methods.
TxnType has one of following values:
 0 – undefined
 1 – Purchase
 2 – Refund

3 – Void

******- *get_TrnStatus(BYTE * pVal), property TrnStatus*

[OUT] *pVal – Get transaction status*

TrnStatus has one of following values:

0 – undefined

1 – approved

2 – declined

3 – reversed

4 – canceled

5.1.3 Подтверждение оплаты

В случае необходимости подтверждения, всегда используется автоподтверждение операций. Это правило также верно для всех методов, для которых в ВРРО требовалось подтверждение.

5.2 Операция «Возврат»

Возврат – операция, обратная операции «Оплата», предназначена для отмены транзакции, даже если она не содержится в текущем пакете транзакций, а была проведена на терминале ранее.

5.2.1 Запрос на возврат:

```
{  
  "method": "Refund",  
  "step": 0,  
  "params": {  
    "amount": "0.30",  
    "discount": "",  
    "merchantId": "0",  
    "rrn": "014721258513",  
    "subMerchant": "i12345678910"  
  }  
}
```

amount - сумма проводимой транзакции,

discount - дополнительная сумма (скидка)

merchantId - индекс мерчанта, который используется для транзакции

rrn – RRN

5.2.2 Ответ

Результат запроса будет аналогичен, ответу на запрос оплаты в 5.1.2, за исключением поля **signVerif**

```
{
  "method": "Refund",
  "step": 0,
  "params": {
    "amount": "0.60",
    "approvalCode": "999999",
    "captureReference": "",
    "cardExpirDate": "2020",
    "cardHolderName": "INSTANT/ISSUE",
    "date": "02.10.2019",
    "discount": "0.00",
    "hstFld63Sf89": "",
    "invoiceNumber": "999999",
    "issuerName": "VISA ПРИВАТ",
    "merchant": "TSTTTTTT",
    "pan": "4731XXXXXXXXX9838",
    "posConditionCode": "00",
    "posEntryMode": "022",
    "processingCode": "000000",
    "receipt": "text-of-receipt",
    "responseCode": "0000",
    "rrn": "99999999999999",
    "rrnExt": "88888888888888",
    "terminalId": "TSTSALE2",
    "time": ""09:11:07",
    "track1": "",
    "txnType": "2",
    "trnStatus": "1",
    "adv": "ПриватБанк.",
    "adv2p": "Беремо і робимо!",
    "bankAcquirer": "ПриватБанк",
    "paymentSystem": "VISA",
    "subMerchant": "i12345678910"
  },
  "error": false,
```

```
"errorDescription": ""  
}
```

5.3 Операция «Отмена»

Операция «Отмена» – предназначена для аннулирования транзакции в рамках текущего пакета транзакций (операционного дня). Доступ к аннулируемой транзакции производится по номеру чека.

5.3.1 Запрос на отмену:

```
{  
  "method": "Withdrawal",  
  "step": 0,  
  "params": {  
    "invoiceNumber": "131220"  
  }  
}
```

5.3.2 Ответ

Результат ответа такой же, как и в п.5.1.2, за исключением поля **signVerif**

```
{  
  "method": "Withdrawal",  
  "step": 0,  
  "params": {  
    "amount": "0.60",  
    "approvalCode": "999999",  
    "captureReference": "",  
    "cardExpiryDate": "2020",  
    "cardHolderName": "INSTANT/ISSUE",  
    "date": "02.10.2019",  
    "discount": "0.00",  
    "hstFld63Sf89": "",  
    "invoiceNumber": "999999",  
  }  
}
```

```

        "issuerName": ""VISA ПРИВАТ",
        "merchant": "TSTTTTTT",
        "pan": "4731XXXXXXXXX983",
        "posConditionCode": "00",
        "posEntryMode": "022",
        "processingCode": "000000",
        "receipt": "text-of-receipt",
        "responseCode": "0000",
        "rrn": "9999999999999",
        "terminalId": "TSTSALE2",
        "time": "09:11:07",
        "track1": "",
        "txnType": "3",
        "trnStatus": "1",
        "adv": "ПриватБанк.",
        "adv2p": "Беремо і робимо!",
        "bankAcquirer": "ПриватБанк",
        "paymentSystem": "VISA",
        "subMerchant": ""
    },
    "error": false,
    "errorDescription": ""
}

```

в методах **Withdrawal** и **WithdrawalPartly** поле **subMerchant** не передаётся, терминал возвращает значение **subMerchant** переданное ранее в методе **Purchase**

5.4 Операция «Частичная отмена»

Операция предназначена для частичной отмены по номеру чека с вводом суммы отмены на кассе.

5.4.1 Запрос на отмену:

```

{
    "method": "WithdrawalPartly",
    "step": 0,
    "params": {

```



```
    "subMerchant": "i12345678910"
  },
  "error": false,
  "errorDescription": ""
}
```

5.5 Операция «Проверка связи»

Касса генерирует сообщение-запрос терминалу. При получении этого запроса, терминал отобразит список мерчантов. Пользователь должен выбрать из списка мерчант, для которого необходимо провести тест связи.

5.5.1 Запрос:

```
{
  "method": "CheckConnection",
  "step": 0
}
```

5.5.2 Ответ:

```
{
  "method": "CheckConnection",
  "step": 0,
  "params": {
    "code": "00", //для совместимости
    "responseCode": "0000"
  },
  "error": false,
  "errorDescription": ""
}
```

5.6 Операция «Чтение банковской карты»

При получении этого запроса, терминал предложит пользователю прокатать карту через магнитный считыватель.

5.6.1 Запрос:

```
{
```

```
"method": "ReadCardBank",  
"step": 0  
}
```

5.6.2 Ответ:

В случае неудачного чтения карты все поля **params** будут пустые

```
{  
  "method": "ReadCardBank",  
  "step": 0,  
  "params": {  
    "hash": "",  
    "track1": "",  
    "track2": "5678*****9876",  
    "track3": ""  
  },  
  "error": false,  
  "errorDescription": ""  
}
```

track1 — первый трек карты в стандарте ISO (необязательное)

track2 — второй трек карты в стандарте ISO

track3 — третий трек карты в стандарте ISO (необязательное)

hash — sha1 хеш номера карты

***track2 необходимо передавать согласно правилам маскирования**

5.7 Операция «Чтение дисконтной карты»

При получении этого запроса, терминал предложит пользователю прокатать карту через магнитный считыватель.

5.7.1 Запрос:

```
{  
  "method": "ReadCardDiscount",  
  "step": 0  
}
```

5.7.2 Ответ:

В случае неудачного чтения карты все поля **params** будут пустые. Описание полей аналогично 5.6.2

```
{  
  "method": "ReadCardDiscount",  
  "step": 0,
```

```
"params": {
    "hash": "",
    "track1": "",
    "track2": "9999999999999939",
    "track3": ""
},
"error": false,
"errorDescription": ""
}
```

5.8 Операция «Версия ПО»

Операция предназначена для получения названия программного обеспечения, установленного на терминал

5.8.1 Запрос:

```
{
    "method": "GetTerminalInfo",
    "step": 0
}
```

5.8.2 Ответ:

```
{
    "method": "GetTerminalInfo",
    "step": 0,
    "params": {
        "version": "TE7E132 TESTKKM000CT22927302"
    },
    "error": false,
    "errorDescription": ""
}
```

в поле version передается версия ПО терминала и мерчант + с/н терминала

5.9 Операция «Ping»

Данная команда не предназначена для буквального “пингования” оборудования на уровне TCP/IP или проверки доступности COM порта. Успешное выполнение данной операции свидетельствует о том, что терминал принимает JSON согласно описываемому протоколу, а также в данный момент свободен и готов к выполнению любого метода.

5.9.1 Запрос:

```
{  
  "method": "PingDevice",  
  "step": 0  
}
```

5.9.2 Ответ:

```
{  
  "method": "PingDevice",  
  "step": 0,  
  "params": {  
    "code": "00", // для совместимости  
    "responseCode" : "0000",  
  },  
  "error": false,  
  "errorDescription": ""  
}
```

5.10 Операция «Баланс»

5.10.1 Запрос:

```
{  
  "method": "GetBalance",  
  "step": 0,  
  "params": {  
    "merchantId": "1" // "0" или "" - вывод всего списка  
  }  
}}
```

5.10.2 Ответ:

```
{  
  "method": "GetBalance",  
  "step": 0,  
  "params": {  
    "balance": "XXX.XX",  
    "code": "00", // для совместимости  
    "responseCode": "0000"
```

```
},  
"error": false,  
"errorDescription": ""  
}
```

При неудаче - "balance": "unknown", "code": "01", "responseCode": "0001"

5.11 Операция «Сервис возврат»

Данная операция является альтернативой операции **«Возврат» (Refund)**. *Примечание:* код сервиса и id мерчанта зашивается хардкодом (см. Табл. 1.1), указывать его не нужно.

5.11.1 Запрос

```
{
  "method": "ServiceRefund",
  "step": 0,
  "params": {
    "amount": "0.30",
    "rrn": "014721258513",
    "subMerchant": "i12345678910"
  }
}
```

Ответ: 5.11.2

```
{  
  "method": "ServiceRefund",  
  "step": 0,  
  "params": {  
    "amount": "0.60",  
    "approvalCode": "999999",  
    "captureReference": "",  
    "cardExpiryDate": "2020",  
    "cardHolderName": "999999999",  
    "date": "02.10.2019",  
    "discount": "0.00",  
    "hstFld63Sf89": "text-of-6389-field",  
    "invoiceNumber": "999999",  
    "issuerName": "INSTANT/ISSUE",  
    "merchant": "TSTTTTTTT",  
    "pan": "4731XXXXXXXXXX9838",
```

```
{
  "posConditionCode": "00",
  "posEntryMode": "022",
  "processingCode": "000000",
  "receipt": "text-of-receipt",
  "responseCode": "0000",
  "rrn": "99999999999999",
  "rrnExt": "88888888888888",
  "terminalId": "TSTSALE2",
  "time": "09:11:07",
  "track1": "",
  "adv": "ПриватБанк.",
  "adv2p": "Беремо і робимо!",
  "bankAcquirer": "ПриватБанк",
  "paymentSystem": "VISA",
  "subMerchant": "i12345678910"
},
"error": false,
"errorDescription": ""
}
```

5.12 Операция «Сервис оплата частями» (Pay by parts)

Примечание: код сервиса и id мерчанта зашивается хардкодом (см. Табл. 1.1), указывать его не нужно.

5.12.1 Запрос

```
{
  "method": "ServicePbP",
  "step": 0,
  "params": {
    "amount": "0.30",
    "amountOfParts": "6",
    "subMerchant": "i12345678910"
  }
}
```

5.12.2 Ответ:

```
{
```

```
"method": " ServicePbP ",
"step": 0,
"params": {
    "amount": "0.60",
    "approvalCode": "999999",
    "captureReference": "",
    "cardExpirDate": "2020",
    "cardHolderName": "INSTANT/ISSUE",
    "date": "02.10.2019",
    "discount": "0.00",
    "amountOfParts": "6",
    "hstFld63Sf89": "text-of-6389-field",
    "invoiceNumber": "999999",
    "issuerName": "VISA ПРИВАТ",
    "merchant": "TSTTTTTT",
    "pan": "4731XXXXXXXXXX9838",
    "posConditionCode": "00",
    "posEntryMode": "022",
    "processingCode": "000000",
    "receipt": "text-of-receipt",
    "responseCode": "0000",
    "rrn": "99999999999999",
    "terminalId": "TSTSALE2",
    "time": "0941",
    "track1": "",
    "adv": "ПриватБанк.",
    "adv2p": "Беремо і робимо!",
    "bankAcquirer": "ПриватБанк",
    "paymentSystem": "VISA",
    "subMerchant": "i12345678910"
},
"error": false,
"errorDescription": ""
}
```

5.13 Операция «сервис возврат оплаты частями» - осуществляет возврат для сервиса ServicePbP.

5.13.1 Запрос:

```
{
  "method": "ServiceRefPbP",
  "step": 0,
  "params": {
    "amount": "350",
    "agreementNum": "12345"
  }
}
```

5.13.2 Ответ:

```
{
  "method": "ServiceRefPbP",
  "step": 0,
  "params": {
    "approvalCode": "999999",
    "captureReference": "",
    "cardExpiryDate": "2020",
    "cardHolderName": "INSTANT/ISSUE",
    "date": "02.10.2019",
    "discount": "0.00",
    "invoiceNumber": "999999",
    "issuerName": ""VISA ПРИВАТ",
    "merchant": "TSTTTTTT",
    "pan": "4731XXXXXXXXX9838",
    "posConditionCode": "00",
    "posEntryMode": "022",
    "processingCode": "000000",
    "receipt": "text-of-receipt",
    "responseCode": "0000",
    "rrn": "99999999999999",
    "terminalId": "TSTSALE2",
    "time": "09:11:07",
    "track1": "",
    "amount": "350",
    "hstFld63Sf89": "Text-of-6389-field",
    "agreementNum": "12345",
  }
}
```

```
    "adv": "ПриватБанк.",  
    "adv2p": "Беремо і робимо!",  
    "bankAcquirer": "ПриватБанк",  
    "paymentSystem": "VISA",  
    "subMerchant": ""  
  },  
  "error": false,  
  "errorDescription": ""  
}
```

5.14 Прочие сервисы

По аналогии с приведенными выше сервисами, вызываются остальные сервисные операции. Коды сервисов и id мерчантов также защищены хардкодом.

- **ServicePbPperiod** (ОЧ в периоде)
- **ServiceRefPbPperiod** (возврат ОЧ в периоде)
- **ServicePartlyRefPbPperiod** (частичный возврат ОЧ в периоде)
- **ServicePartlyRefPbP** (частичный возврат ОЧ)
- **ServiceInstantPbl** (мгновенная рассрочка) // Pay by installment
- **ServiceRefPbl** (возврат МР)
- **ServicePartlyRefPbl** (частичный возврат МР)
- **ServicePblAct** (мгновенная рассрочка акция)
- **ServiceRefPblAct** (возврат МР акция)
- **ServicePartlyRefPblAct** (частичный возврат МР акция)

Отличие состоит в названии метода, а также в варьируемом параметре – **agreementNum**, либо **amountOfParts**

Таблица 1.1 - Хардкодные номера сервисов и id мерчантов:

Service ID	Service name	Описание	MerchantID
054	ServiceRefund	Сервис возврат	5
046	ServicePbP	Сервис оплата частями (ОЧ)	6
062	ServiceRefPbP	Сервис оплата частями возврат	6
047	ServicePartlyRefPbP	Сервис оплата частями частичный возврат	6
044	ServicePbPperiod	Сервис оплата частями в периоде	3
045	ServiceRefPbPperiod	Сервис оплата частями в периоде возврат	3
061	ServicePartlyRefPbPperiod	Сервис оплата частями в периоде частичный возврат	3
046	ServiceInstantPbl	Сервис мгновенная рассрочка Pay by installment	4
062	ServiceRefPbl	Сервис мгновенная рассрочка Pay by installment возврат	4
047	ServicePartlyRefPbl	Сервис мгновенная рассрочка Pay by installment частичный возврат	4
046	ServicePblAct	Сервис мгновенная рассрочка акция	7
062	ServiceRefPblAct	Сервис мгновенная рассрочка акция возврат	7
047	ServicePartlyRefPblAct	Сервис мгновенная рассрочка акция частичный возврат	7

5.15 Универсальный сервис. Поскольку теоретически количество сервисных операций может быть велико (**J мерчанты**), нет возможности предусмотреть уникальный метод для каждой из них. Для решения данной задачи и существует универсальный сервис. В отличие от уникальных сервисов, представленных выше, универсальный сервис требует указать не только номер мерчанта, но и номер сервиса, а также может принимать более одного параметра, для чего их перечень необходимо указать через slash (/).

5.15.1 Запрос:

```
{
  "method": "ServiceGeneric",
  "step": 0,
  "params": {
    "amount": "0.01",
```

```
        "param": "6", // если их более 1, тогда записываем так: 1/2/3/N
        "srvNum": "046",
        "merchantId": "6"
    }
}
```

5.15.2 Ответ:

```
{
  "method": "ServiceGeneric",
  "step": 0,
  "params": {
    "responsCode": "0000",
    "amount": "350",
    "cardExpiryDate": "2020",
    "receipt": "Text-of-receipt",
    "hstFld63Sf89": "Text-of-63-89-field",
    "agreementNum": "12345", // либо rrn, либо amountOfParts
    "adv": "ПриватБанк.",
    "adv2p": "Беремо і робимо!",
    "bankAcquirer": "ПриватБанк",
    "paymentSystem": "VISA"
  },
  "error": false,
  "errorDescription": ""
}
```

5.16 Cashback. Операция продажи с обналичиванием средств. По своей структуре фактически повторяет логику операции Purchase (продажа). В сравнении с **Purchase**, отсутствуют возвращаемые параметры **signVerif**, **tnxType**, **trnStatus**. Добавляется параметр **result** и **resultCashback**.

5.16.1 Запрос

```
{
  "method": "Cashback",
  "step": 0,
  "params": {
    "amount": "0.30",
    "amountCash": "1.00",
```

```
        "merchantId": "1",
        "subMerchant": "i12345678910"
    }
}
```

5.16.2 Ответ

```
{
  "method": "Cashback",
  "step": 0,
  "params": {
    "amount": "1.30",
    "amountCash": "1.00",
    "approvalCode": "527080",
    "invoiceNumber": "139",
    "merchant": "1",
    "receipt": "text-of-receipt",
    "result": "Операція успішна",
    "resultCashback": "true", // false
    "rrn": "014721391803",
    "rrnExt": "88888888888888",
    "captureReference": "",
    "cardExpiryDate": "2020",
    "cardHolderName": "INSTANT/ISSUE",
    "date": "02.10.2019",
    "discount": "0.00",
    "hstFld63Sf89": "",
    "issuerName": ""VISA ПРИВАТ",
    "merchant": "TSTTTTTT",
    "pan": "4731XXXXXXXXX9838",
    "posConditionCode": "00",
    "posEntryMode": "022",
    "processingCode": "000000",
    "responseCode": "0000",
    "terminalId": "TSTSALE2",
    "time": "09:11:07",
    "track1": "",
    "adv": "ПриватБанк.",
    "adv2p": "Беремо і робимо!",
  }
}
```

```

        "bankAcquirer": "ПриватБанк",
        "paymentSystem": "VISA",
        "subMerchant": "i12345678910"
    },
    "error": false,
    "errorDescription": ""
}

```

result: “Операція успішна” – параметр, соответствующий полю <Result></Result> аналогичного сценария BPOS в реализации Ingenico.

resultCashback true/false – параметр на случай частичного апрува операции, когда оплата произошла, а кэшбек – нет.

amountCash - сумма выдачи наличных

5.17 Audit. Он же X - balance

5.17.1 Запрос

```

{
    "method": "Audit",
    "step": 0,
    "params": {
        "merchantId": "0"
    }
}

```

5.17.2 Ответ

```

{
    "method": "Audit",
    "step": 0,
    "params": {
        "receipt": "08:50:00 03/10/2019 ===== [ X БАЛАНС ]
===== ТЕРМ. TESTKKM0 B:000007 for slip: 0001 0067
Оператор MASTER ПРИВАТ бонуси/знижки 0.00 ГРН без бонусів/знижок 1.01
ГРН з бонусами/знижками 1.01 ГРН VISA ПРИВАТ бонуси/знижки 0.00 ГРН
без бонусів/знижо 0.05 ГРН з бонусами/знижками 0.05 ГРН
===== Загальні підсумки: бонуси/знижки 0.00 ГРН без
бонусів/знижок 1.06 ГРН з бонусами/знижками 1.06 ГРН
===== Ingenico Group TE7E v.131.403 15/03/ P9 ",
        "responseCode": "0000"
    }
}

```

```

    },
    "error": false,
    "errorDescription": ""
}

```

5.18 Сверка (Загальний звіт) Предназначена для инициирования и отправки итоговых сумм на хост для сверки с данными, имеющимися на хосте. Терминал может выполнить сверку итогов поочередно по всем мерчантам, либо по каждому отдельно – в зависимости от наличия поля Merchant ID.

5.18.1 Запрос

```

{
  "method": "Verify",
  "step": 0,
  "params": {
    "merchantId": "0"
  }
}

```

5.18.2 Ответ (на примере BPOS Ingenico)

```

{
  "method": "Verify",
  "step": 0,
  "params": {
    "receipt": "07:53:00 02/10/2019 ===== [ Z БАЛАНС ]
===== ТЕРМ. TESTKKM0 B:000006 for slip: 0001 0061
Оператор VISA ПРИВАТ бонуси/знижки 0.00 ГРН без бонусів/знижок 0.03 ГРН
з бонусами/знижками 0.03 ГРН ===== Загальні
підсумки: бонуси/знижки 0.00 ГРН без бонусів/з нижок 0.03 ГРН з
бонусами/знижками 0.03 ГРН ===== ПІДСУМКИ
ВИЛУЧЕНІ ===== Ingenico Group TE7E v.131.403 15/03/
P9 ",
    "responseCode": "0000"
  },
  "error": false,
  "errorDescription": ""
}

```

5.18.3 Ошибки

```

{
  "method": "Verify",

```

```

"step": 0,
"params": {
    "receipt": "",
    "responseCode": "0001"
},
"error": false,
"errorDescription": "Cannot obtain receipt: log file is empty!"
}

```

5.19 Копия чека «Сверка» (Загальний звіт). Помимо передачи чека на кассу, также производится печать чека терминалом.

5.19.1 Запрос

```

{
  "method": "VerifyCopy",
  "step": 0,
  "params": {
    "merchantId": "0"
  }
}

```

5.19.2 Ответ

```

{
  "method": "VerifyCopy",
  "step": 0,
  "params": {
    "receipt": "КОПИЯ 12:21:00 27/08/2019 ===== [ Z
БАЛАНС ] ===== ТЕРМ. TESTKKM0 B:000001 for slip: 0001
0011 Оператор VISA ПРИВАТ бонуси/знижки 0.01 ГРН без бонусів/знижок
0.03 ГРН з бонусами/знижками 0.04 ГРН ===== Загальні
підсумки: бонуси/знижки 0.01 ГРН без бонусів/знижок 0.03 ГРН з
бонусами/знижками 0.04 ГРН ===== КОПИЯ 07:53:00
02/10/2019 ===== [Загальний звіт ]
===== ТЕРМ. TESTKKM0 B:000006 for slip: 0001 0061
Оператор VISA ПРИВАТ бонуси/знижки 0.00 ГРН без бонусів/знижок 0.03 ГРН
з бонусами/знижками 0.03 ГРН ===== Загальні
підсумки: бонуси/знижки 0.00 ГРН без бонусів/знижок 0.03 ГРН з

```

бонусами/знижками 0.03 ГРН ===== Ingenico Group
TE7E v.131.403 15/03/ P9 ",
 "responseCode": "0000"
},
"error": false,
"errorDescription": ""
}

5.20 Получение чека - PrintReceiptNum . Распечатывается ли чек терминалом в данном случае регулируется специальным флагом.

- prnFlag = 0 чек терминалом не распечатывается, выдаётся на кассу
- prnFlag = 1 печать чека терминалом, параллельно с выдачей на кассу
- invoiceNumber = "0" или "" - по умолчанию берётся номер последнего чека в пакете
- Параметр "invoiceNumber", отличающийся от "0" или "" является явным указанием номера чека, запрашиваемого кассой. То есть, если не передаётся номер чека вручную, то метод работает как печать последнего чека.

5.20.1 Запрос

```
{  
  "method": "PrintReceiptNum",  
  "step": 0,  
  "params": {  
    "invoiceNumber": "123",  
    "prnFlag": "0"  
  }  
}
```

5.20.2 Ответ

```
{  
  "method": "PrintReceiptNum",  
  "step": 0,  
  "params": {  
    "receipt": "ОПЛАТА СУМА 0.01 ГРН Підпис власника картки не потрібен  
MasterCard XXXXXXXXXXXX9716 AID: A0000000041010 ТЕРМІНАЛ TESTKKMO  
03/10/2019 09:04:40 КОД АВТОРИЗАЦІЇ: 982088 ЧЕК N: 68 RRN: 014721392474  
EXTERNAL RRN: Ingenico Group TE7E v.131.403 15/03/ P9 ",  
    "responseCode": "0000"  
  },  
  "error": false,
```

```
"errorDescription": ""
}
```

5.20.3 Ошибки

В случае ошибки, на кассу возвращается параметр `responseCode >= 1000`, пустой параметр `receipt` и описание ошибки в `errorDescription`.

```
{
  "method": "PrintReceiptNum",
  "step": 0,
  "params": {
    "receipt": "",
    "responseCode": "1000"
  },
  "error": true,
  "errorDescription": "Cannot obtain receipt: Неверный мерчант!"
}
```

Примечание: чеки должны накапливаться в памяти терминала в виде некоего организованного хранилища, будь то перечень специально пронумерованных файлов, либо СУБД, либо текстовый лог. Понятие **последний чек** - это есть результат последней операции выполненной терминалом в пакете. Результат печати чека по номеру, Z, X баланса или их копий не заменяет собой понятие последнего чека.

5.21 Печать информации обо всех транзакциях в пакете. Помимо передачи чека на кассу, также производится печать чека терминалом.

5.21.1 Запрос

```
{
  "method": "PrintBatchJournal",
  "step": 0,
  "params": {
    "merchantId": "0"
  }
}
```

`merchantId` - индекс мерчанта, который будет использован для данной операции
Если `merchantId` = "0" или "", производится обработка данных по всем доступным мерчантам

5.21.2 Ответ

```
{
  "method": "PrintBatchJournal",
```

```
"step": 0,
"params": {
    "receipt": "text-of-receipt",
    "responseCode": "0000"
},
"error": false,
"errorDescription": ""
}
```

5.21.3 Ошибки

В случае ошибки, на кассу возвращается параметр `responseCode` ≥ 1000 , пустой параметр `receipt` и описание ошибки в `errorDescription`.

```
{
    "method": "PrintBatchJournal",
    "step": 0,
    "params": {
        "receipt": "",
        "responseCode": "1000"
    },
    "error": true,
    "errorDescription": "Cannot obtain receipt: Неверный мерчант!"
}
```

6. Служебные сообщения

Service Messages – особый тип сообщений, которые обрабатываются сервером асинхронно, т. е. не зависимо от текущей операции или состояния. Используются для передачи служебных команд от клиента серверу и, наоборот, для получения клиентом сообщений о состоянии от сервера.

6.1 Сообщение deviceBusy - данное сообщение сервер отправляет клиенту в ответ на любое не сервисное сообщение в том случае, если сервер занят текущей операцией и не может принять следующий запрос. Ответ на это сообщение не требуется. Генерируется только терминалом.

```
{  
  "method": "ServiceMessage",  
  "step": 0,  
  "params": {  
    "msgType": "deviceBusy"  
  },  
  "error": false,  
  "errorDescription": ""  
}
```

6.2 Сообщение interrupt – требование прерывания инициированной операции.

```
{  
  "method": "ServiceMessage",  
  "step": 0,  
  "params": {  
    "msgType": "interrupt"  
  }  
}
```

6.2.1 Ответ на сообщение interrupt – подтверждение interruptTransmitted. Генерируется только терминалом.

```
{  
  "method": "ServiceMessage",  
  "step": 0,  
  "params": {
```

```
    "msgType": " interruptTransmitted"
  },
  "error": false,
  "errorDescription": ""
}
```

6.3 Сообщение methodNotImplemented – сообщение генерируется терминалом в ответ на запрос не поддерживаемого метода.

```
{
  "method": "ServiceMessage",
  "step": 0,
  "params": {
    "msgType": " methodNotImplemented"
  },
  "error": false,
  "errorDescription": ""
}
```

6.4 Сообщение getMerchantList – запрос клиентом у сервера terminal software version + terminal profile ID + POS S/N + the list of acquirers.

6.4.1 Запрос:

```
{
  "method": "ServiceMessage",
  "step": 0,
  "params": {
    "msgType": "getMerchantList"
  }
}
```

6.4.2 Ответ на данное сообщение:

```
{
```

```
"method": "ServiceMessage",
"step": 0,
"params": {
    "3": "Оплата Частями в периоде",
    "4": "Оплата Частями",
    "5": "Мгновенная Рассрочка",
    "6": "Мгновенная Рассрочка Акция",
    "12": "Табелирование",
    "msgType": "getMerchantList"
},
"error": false,
"errorDescription": ""
}
```

6.4 Сообщение getMaskList – запрос клиентом у сервера terminal software version + terminal profile ID + POS S/N + the list of merchants.

6.4.1 Запрос:

```
{
"method": "ServiceMessage",
"step": 0,
"params": {
    "msgType": " getMaskList"
}
}
```

6.4.2 Ответ на данное сообщение:

```
{
"method": "ServiceMessage",
"step": 0,
"params": {
    "1": "TESTKKM0",
    "2": "TESTKKM1",
    "3": "X1110B6L",
    "4": "X11113W2",
    "5": "X10D2QLB",
    "6": "X11193W2",

```

```

    "7": "REFUND3",
    "8": "REFUND2",
    "9": "TESTKKM2",
    "10": "INKASS1",
    "11": "INKASS2",
    "12": "S1KITV99",
    "msgType": "getMaskList"
  },
  "error": false,
  "errorDescription": ""
}

```

6.5 Сервисное сообщение debug – используется для отключения/включения функции непосредственного снятия логов портов COM, USB, ETH. Файл, в который производится запись лога, именуется trace.log и создаётся в каталоге, текущем по отношению к инициировавшему логирование ПО. **Данный функционал полностью реализуется на стороне клиента (запрос, ответ, реализация записи в файл).**

Формат лога:

[17/10/2019 15:27:43] Written data (COM9)

```

7b 22 6d 65 74 68 6f 64 22 3a 22 50 69 6e 67 44
65 76 69 63 65 22 2c 22 73 74 65 70 22 3a 30 7d
00

```

```

{"method":"PingDevice","step":0}.

```

[17/10/2019 15:27:43] Read data (COM9)

```

7b 22 6d 65 74 68 6f 64 22 3a 22 50 69 6e 67 44
65 76 69 63 65 22 2c 22 73 74 65 70 22 3a 30 2c
22 70 61 72 61 6d 73 22 3a 7b 22 63 6f 64 65 22
3a 22 30 30 22 2c 22 72 65 73 70 6f 6e 73 65 43
6f 64 65 22 3a 22 30 30 30 30 22 7d 2c 22 65 72
72 6f 72 22 3a 66 61 6c 73 65 2c 22 65 72 72 6f
72 44 65 73 63 72 69 74 70 74 69 6f 6e 22 3a 22
22 7d 00

```

```

{"method":"PingDevice",
"step":0,
"params":{"code":"00",
"responseCode":"0000"},
"error":false,
"errorDescription":""}.

```

[17/10/2019 15:27:44] - Close port COM9

6.5.1 Запрос:

```

{
  "method": "ServiceMessage",
  "step": 0,
  "params": {
    "msgType": "debug"
  }
}

```

6.5.2 Ответ на данное сообщение:

```
{
  "method": "ServiceMessage",
  "step": 0,
  "params": {
    "msgType": "debugOn" // либо debugOff
  },
  "error": false,
  "errorDescription": ""
}
```

6.6 Service message getLastResult

Параметр LastResult используется для получения результата выполняемой транзакции
Перечень возможных значений соответствует аналогичному property BPOS

get_LastResult(BYTE pVal), property LastResult*
Gets the result of fulfilling of the last transaction
[OUT] pVal- Returned values:
0 – successfully fulfilled
2 – in progress

```
{
  "method": "ServiceMessage",
  "step": 0,
  "params": {
    "msgType": "getLastResult"
  }
}
```

6.6.1 getLastResult ответ

```
{
  "method": "ServiceMessage",
  "step": 0,
  "params": {
    "msgType": "getLastResult",
    "LastResult": "0"
  }
}
```

```

    },
    "error": false,
    "errorDescription": ""
}

```

6.7 Service message getLastStatMsgCode

Параметр LastStatMsgCode указывает текущий ход выполнения операции терминалом. Обычно данный параметр изменяется параллельно смене событий в интерфейсе устройства, при взаимодействии с кассиром/пользователем. Перечень возможных значений соответствует аналогичному property BPOS.

get_LastStatMsgCode(BYTE pVal), property LastStatMsgCode*
 [OUT] pVal - returns one following status codes:

- 0 – status code is not available.
- 1 – card was read
- 2 – used a chip card
- 3 – authorization in progress
- 4 – waiting for cashier action
- 5 – printing receipt
- 6 – pin entry is needed
- 7 – card was removed
- 8 – EMV multi aid's
- 9 – waiting for card
- 10 – in progress
- 11 – correct transaction

```

{
  "method": "ServiceMessage",
  "step": 0,
  "params": {
    "msgType": "getLastStatMsgCode"
  }
}

```

6.7.1 getLastStatMsgCode ответ

```

{
  "method": "ServiceMessage",
  "step": 0,
  "params": {
    "msgType": "getLastStatMsgCode",
    "LastStatMsgCode": "6"
  },

```

```
"error": false,  
"errorDescription": ""  
}
```

6.8 Service message getLastStatMsgDescription

Параметр LastStatMsgDescription указывает текущий ход выполнения операции терминалом. Обычно данный параметр изменяется параллельно смене событий в интерфейсе устройства, при взаимодействии с кассиром/пользователем. Перечень возможных значений соответствует аналогичному property BPOS и является текстовой расшифровкой значений параметра LastStatMsgCode.

get_LastStatMsgCode(BYTE pVal), property LastStatMsgCode*

[OUT] pVal - returns one following status codes:

0 – status code is not available.

1 – card was read

2 – used a chip card

3 – authorization in progress

4 – waiting for cashier action

5 – printing receipt

6 – pin entry is needed

7 – card was removed

8 – EMV multi aid's

9 – waiting for card

10 – in progress

11 – correct transaction

```
{  
  "method": "ServiceMessage",  
  "step": 0,  
  "params": {  
    "msgType": "getLastStatMsgDescription"  
  }  
}
```

6.8.1 getLastStatMsgDescription ответ

```
{  
  "method": "ServiceMessage",  
  "step": 0,  
  "params": {  
    "msgType": "getLastStatMsgDescription",  
    "statMsgCode": 1,  
    "statMsgDescription": "Card read successfully"  
  }  
}
```

```
        "LastStatMsgDescription": "pin entry is needed"
    },
    "error": false,
    "errorDescription": ""
}
```

6.9 Identify Идентификация принадлежности данного терминала и версии его ПО АТ КБ ПриватБанк, а также информация о производителе и модели данного терминала.

6.9.1 Запрос

```
{
    "method": "ServiceMessage",
    "step": 0,
    "params": {
        "msgType": "identify"
    }
}
```

6.9.2 ответ

```
{
    "method": "ServiceMessage",
    "step": 0,
    "params": {
        "msgType": "identify",
        "result": "true",
        "vendor": "PAX",
        "model": "s800"
    },
    "error": false,
    "errorDescription": ""
}
```

6.10 Service message getDiscountName используется в рамках процедуры корректировки транзакции. Вызывается в случае getLastStatMsgCode = 11.

6.10.1 Запрос

```
{
  "method": "ServiceMessage",
  "step": 0,
  "params": {
    "msgType": "getDiscountName"
  }
}
```

6.10.2 ответ

В случае вызова методов вне предусмотренного контекста или отсутствия параметров передавать следующие (default) значения:

- **getDiscountName** возвращается значение "discountName": "NO_D_GROUP"
- **pan** возвращается значение "pan": "PAN_UNDEF"
- **posEntryMode** возвращается значение "posEntryMode": "MODE_UNDEF"

```
{
  "method": "ServiceMessage",
  "step": 0,
  "params": {
    "msgType": "getDiscountName",
    "discountName": "someDiscountGroup",
    "pan": "123456*****9876",
    "posEntryMode": "071"
  },
  "error": false,
  "errorDescription": ""
}
```

Правила маскирования PAN:

Для терминалов PAX:

- если параметр `mask_pan = true` - передаются первые 6 и последние 4 цифры номера карты, другие маскированы звёздочками (*)
- если параметр `mask_pan = false` - необходимо маскировать PAN согласно требованиям платежных систем (по стандарту)

Для терминалов Ingenico, Verifone:

- при добавлении мерчанта в исключения и подключении соответствующих комиссий - передаются первые 6 и последние 4 цифры номера карты, другие маскированы звёздочками (*)
- для остальных терминалов сети - необходимо маскировать PAN согласно требованиям платежных систем (по стандарту)

6.11 Service message correctTransaction - корректировка транзакции в рамках операции **Purchase**.

```
{
  "method": "ServiceMessage",
  "step": 0,
  "params": {
    "msgType": "correctTransaction",
    "amount": "0.02",
    "dicsount": "0.01"
  }
}
```

Ошибки:

В случае некорректной установки значений(not a number, delimiter != , | | . etc.) коррекция не применяется,

"error": true, "errorDescription": "Корекція неможлива"

Purchase продолжает выполнение с исходной суммой

Остальные ошибки приходят в основном ответе хоста на Purchase

6.11.1 Service message correctionTransmitted - ответное сообщение в случае успешного применения корректировки транзакции.

```
{
  "method": "ServiceMessage",
  "step": 0,
  "params": {
    "msgType": "correctionTransmitted"
  },
  "error": false,
  "errorDescription": ""
}
```

Важно. Correct Transaction - general workflow

1. Вызов базового Purchase
2. Вызов service message getLastStatMsgCode if 11 ->
3. Получаем имя группы дисконта посредством getDiscountName
4. Касса анализирует группу дисконта, если принимает решение о скидке ->
5. Производится коррекция транзакции с учётом выбранной скидки путём вызова correctTransaction (карта, PIN, etc. повторно НЕ запрашиваются)
6. В случае успешной передачи коррекции, в пределах установленного Adjust Timeout касса принимает serviceMessage correctionTransmitted
7. В противном случае, Purchase продолжает выполняться в рамках исходных (первоначальных) данных.

Патчноут описания протокола

v.1.0.0.1-1.0.0.3 - исправление мелких неточностей

v.1.0.0.4 - уточнение, в качестве значений того или иного поля JSON все не-ASCII символы в т. ч.

кириллица, передаются в кодировке utf-8

v.1.0.0.5 - добавлено описание свойств протокола BPOS приведённых в качестве примеров

(get_LastResult, get_LastErrorCode и пр.)

v.1.0.0.6 - убраны методы getLastErrorCode и getLastErrorDescription, добавлено уточнение

касательно id мерчантов по сервисным методам

v.1.0.0.7 - в сервисе "ServicePbP" добавлено поле "amountOfParts" в ответе терминала;

- уточнение принимаемого формата данных в поле "amount" (п.6.1.)

v.1.0.0.8 - добавлены методы CashAdvance и PrintHTML (п.5.22, 5.23)

v.1.0.0.9 - уточнение по методам "ServiceRefund", "ServicePbP", по именным сервисам нет необходимости передавать поле "merchantId", т.к. эти данные зашиты в методах хардкодом (см. Таблица 1.1)

v.1.0.1.0 - добавлено описание особенностей genericDriverJsonUSB_Linux

v.1.0.1.1 - убрана передача track2 при проведении всех операций, кроме методов **ReadCardBank** и **ReadCardDiscount**

v.1.0.1.2 - в методе "**Purchase**" добавлено поле "**facepay**" для запуска оплаты через FacePay24 с кассы

- в методе "**CashAdvance**" убраны поля "currencyCode" и "accountNumber"

v.1.0.1.3 - уточнение по работе с вещественными числами в методе "**Purchase**"

v.1.0.1.4 - для финансовых операций добавлены поля "**adv**", "**adv2p**" и "**bankacquirer**"

v.1.0.1.5 - в п.2 добавлено пошаговое описание проведения хендшейка со стороны кассового ПО, дополнено описание канального протокола

v.1.0.1.6 - исправлена опечатка в поле "**errorDescription**" возвращаемом терминалом

- исправлен запрос метода "**GetBalance**"

- добавлены методы "**getDiscountName**" (п.6.10) и "**correctTransaction**" (п.6.11)

v.1.0.1.7 - расширение метода correctTransaction

- исправлена опечатка "bankaquirer" на "bankAcquirer"

- добавлено поле в ответе терминала "paymentSystem"

v.1.0.1.8 - добавление поля "rrnExt"

- изменение правил передачи track2 в методе ReadCardBank
- при responseCode отличных от 0000 терминал передаёт "error":true

v.1.0.1.9 - добавлено поле в ответе терминала по финансовым операциям "panHash"

v.1.0.2.0 - добавлено поле в ответе терминала по финансовым операциям "subMerchant"

v.1.0.2.1 - убрали поле "panHash"

- добавлено поле "subMerchant" в запросе по фин. операциям
- z-отчёт переименован в Загальний звіт